centro universitário FEI

CC1620 - Desenvolvimento Web
CCP120 - Programação Full Stack

Não esqueçam de marcar presença!

Servidor


COMO FUNCIONA?
A INTERNET

Introdução à Internet

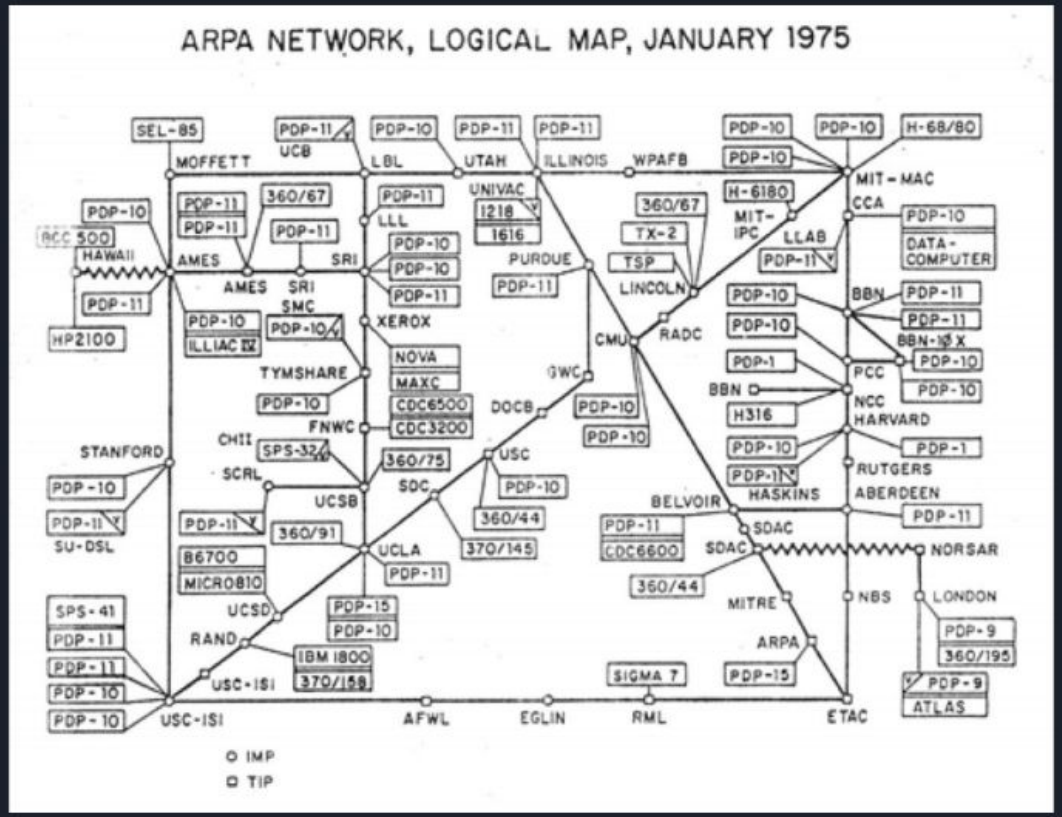
- História da ArpaNet e Internet
- Arquitetura da Internet
- Redes
- Protocolo HTTP

ArpaNet

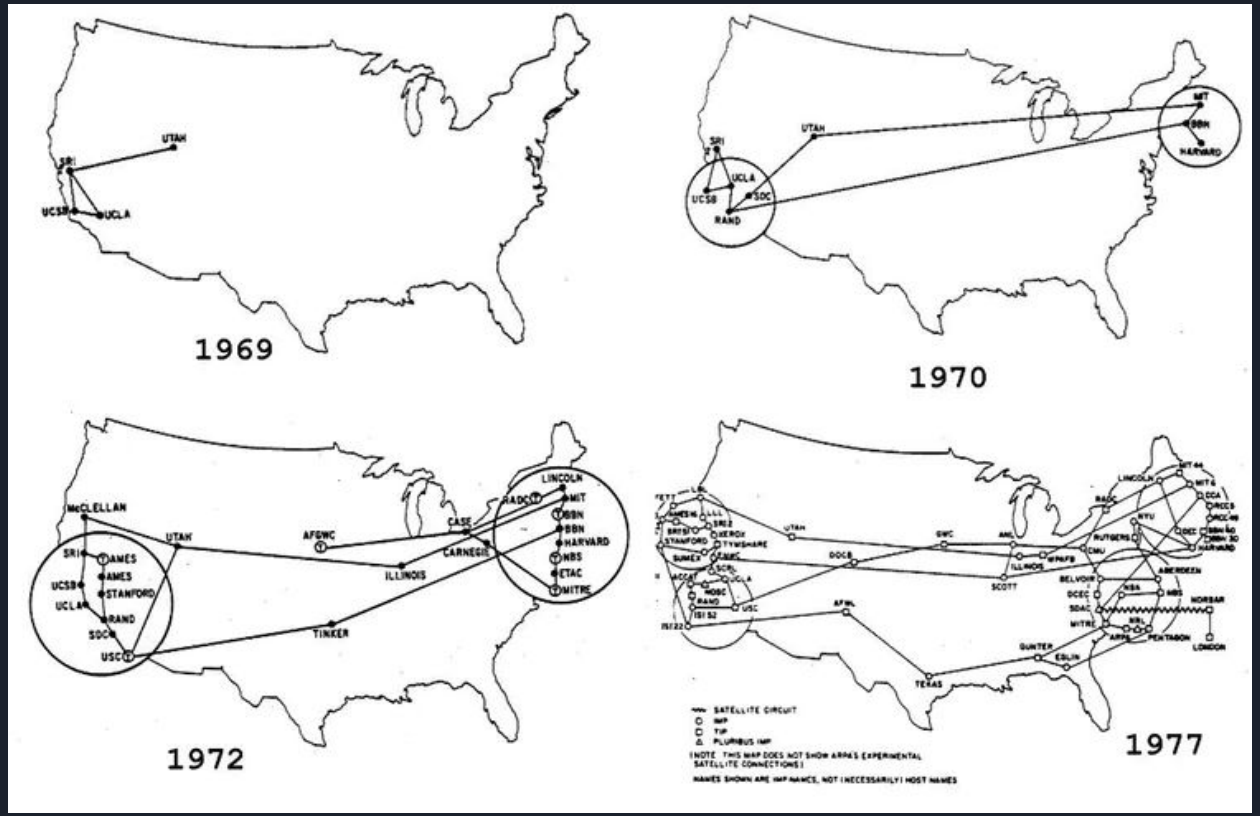
- Pode ser considerada a precursora da Internet
- Criada em 1969 pela ARPA (Advanced Research Project Agency)
- Objetivo:
  - Interligar bases militares e os departamentos de pesquisa nos EUA
- Financiada durante a Guerra Fria


A HISTÓRIA
DA INTERNET

ArpaNet


ARPA NETWORK, LOGICAL MAP, JANUARY 1975
SEL-85
MOFFETT
PDP-11
UCB
PDP-10
LBL
PDP-11
UTAH
PDP-11
ILLINOIS
WPAFB
PDP-10
PDP-10
H-68/80
MIT-MAC
PDP-10
CCA
DATA-COMPUTER
PDP-10
ACC 500
HAWAII
PDP-11
HP2100
AMES
SRI
360/67
PDP-11
PDP-11
PDP-11
LLL
PDP-10
PDP-10
PDP-11
UNIVAC
1218
1616
PURDUE
PDP-11
360/67
TX-2
TSP
LINCOLN
H-6180
MIT-IPC
LLAB
PDP-11
BBN
PDP-11
PDP-11
BBN-1822
PDP-10
PCC
PDP-10
NCC
HARVARD
PDP-1
RUTGERS
ABERDEEN
PDP-11
AMES
SRI
SMC
PDP-10
ILLIAC IV
PDP-10
XEROX
NOVA
MAXC
CDC6500
CDC3200
TYMSHARE
PDP-10
FNWC
CMU
RADC
OWC
DOCB
PDP-10
PDP-10
PDP-10
PDP-10
BBN
H316
PDP-10
PDP-11
HASKINS
BELVOIR
PDP-11
CDC6600
SDAC
SDAC
NORSAR
NBS
LONDON
PDP-9
360/195
PDP-9
ATLAS
360/44
MITRE
ARPA
PDP-15
SIGMA 7
CHII
SPS-32
360/75
USC
PDP-10
360/44
370/145
STANFORD
PDP-10
PDP-11
SU-DSL
SCRL
UCSB
SDC
PDP-11
360/91
UCLA
PDP-11
B6700
MICRO810
SPS-41
PDP-11
PDP-11
PDP-10
PDP-10
UCSD
PDP-15
PDP-10
RAND
IBM 1800
370/158
USC-ISI
USC-ISI
AFWL
EGLIN
RML
ETAC
IMP
TIP

ArpaNet


UTAH
SRI
UCSB
UCLA
1969
UTAH
SRI
UCLA
SDC
UCSB
RAND
MIT
BBN
HARVARD
1970
McCLELLAN
UTAH
AFGWC
CASE
CARNEGIE
ILLINOIS
RADC
LINCOLN
MIT
BBN
BBN
HARVARD
NBS
RETAC
MITRE
SRI
AMES
AMES
STANFORD
UCSB
UCLA
RAND
SDC
USC
TINKER
1972
SATELLITE CIRCUIT
IMP
TIP
PLURIBUS IMP
(NOTE: THIS MAP DOES NOT SHOW ARPA'S EXPERIMENTAL SATELLITE CONNECTIONS)
NAMES SHOWN ARE IMP NAMES, NOT (NECESSARILY) HOST NAMES
UTAH
SCOTT
TEXAS
GUNTER
ERLIN
LINCOLN
RADC
RUTGERS
CMU
ABERDEEN
BELVOIR
SDAC
MITRE
PENTAGON
LONDON
NORSAR
1977

ArpaNet

- Em apenas 4 anos a ArpaNet atingiu cerca de 50 milhões de pessoas
- Em 1984, A Guerra Fria estava no seu fim e a ArpaNet foi dividida em dois grupos:
  - **MILNET: rede militar**
  - **ARPANET: rede não-militar**

ArpaNet

- A ARPANET passou a ser estudada por alunos de diversas universidades



A internet

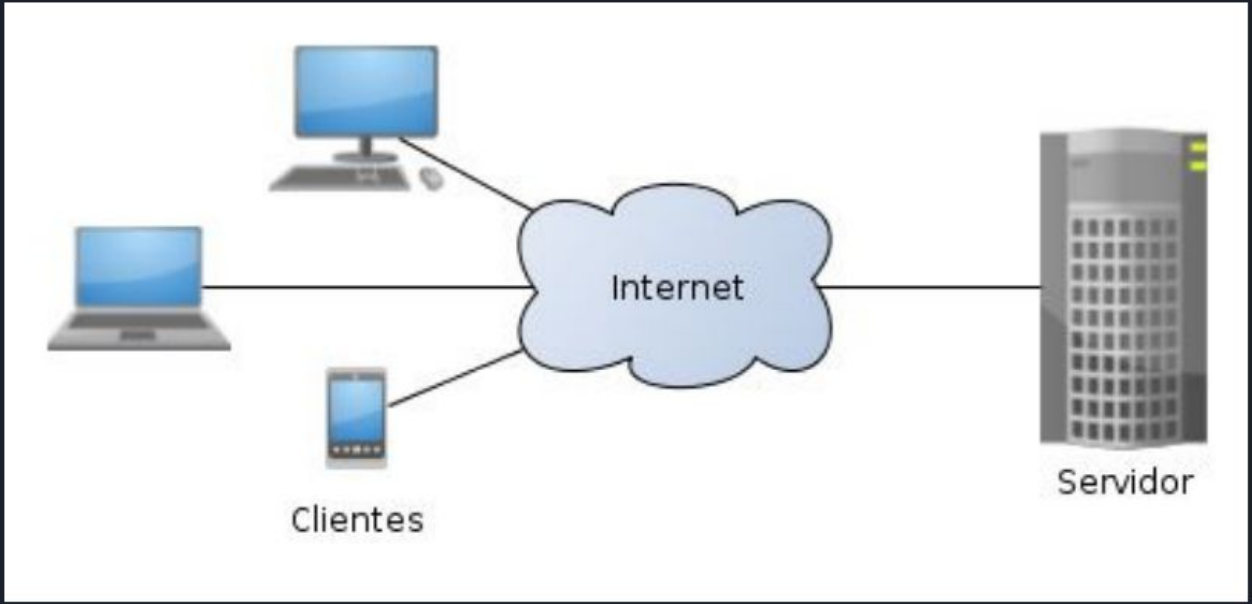
Internet

- Internet como conhecemos hoje é na maior parte o que se chama WWW (World Wide Web)
- WWW é um conjunto de documentos e ligações com informações multimídia
- Contudo, Internet não é só WWW e sim qualquer forma de comunicação entre computadores geograficamente separados que utiliza a rede pública.

Redes

- A WEB é baseada na arquitetura Cliente/Servidor
- Os navegadores (browsers) podem ser os clientes
- E as informações podem estar centralizadas em um único computador: servidor
- Este computador (servidor) guarda todas as informações que são utilizadas pelos clientes

Modelo Cliente/Servidor da Web



Modelo Cliente/Servidor da Web

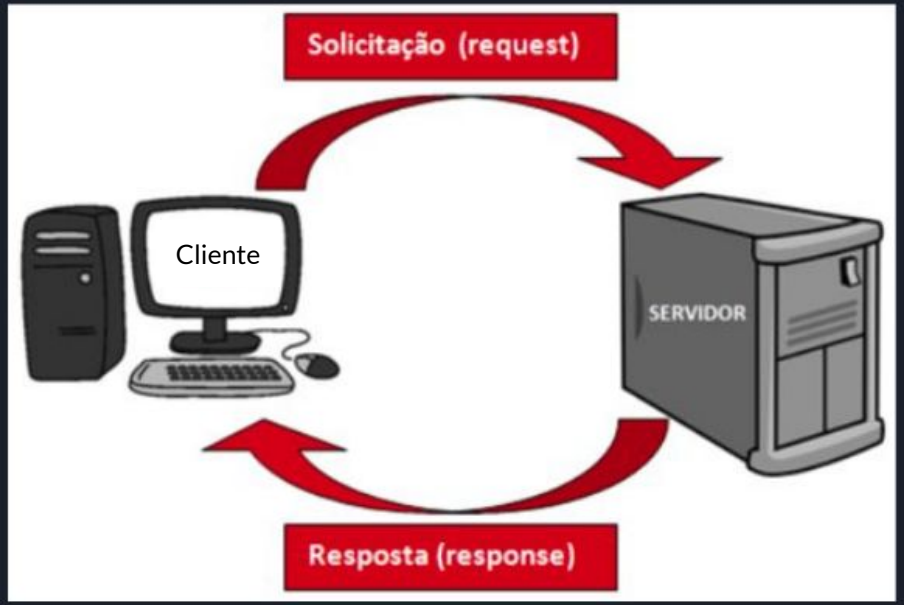
Para que uma rede funcione, é preciso estabelecer alguns conceitos:

- Protocolo: conjunto de regras para a "conversação"
- Meio de Transmissão: canal onde a comunicação ocorre
- Controle de Acesso ao Meio: regra para acessar o canal de comunicação
- Entidades conectadas: computadores envolvidos

Protocolo de comunicação da Web

- O protocolo de comunicação utilizado na WEB é o HTTP (*HyperText Transfer Protocol*)
- Essa é a maneira como se troca mensagens entre servidores e clientes
- Neste protocolo, o cliente (navegador) envia uma solicitação para o servidor pedindo qual é o recurso que está interessado.
- O servidor, então, envia uma resposta ao cliente

Protocolo de comunicação da Web



Protocolo de comunicação da Web

- Exemplo de requisição HTTP:

```
GET / HTTP/1.1
Host: spesa.com.br
User-Agent: Mozilla/5.0 (X11; U; Linux i686; en-US) Gecko/20061201 Firefox/2.0.0.3 (Ubuntu-feisty)
Accept: text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,text/plain;q=0.8,image/png
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
```

Protocolo de comunicação da Web

- Exemplo de resposta HTTP:

```
HTTP/1.x 200 OK
Date: Fri, 04 May 2007 16:05:43 GMT
Server: Apache/2.0.59 (Unix) mod_ssl/2.0.59 OpenSSL/0.9.7a DAV/2 PHP/4.4.4 mod_bwlimited/1.4
Cache-Control: no-cache
Keep-Alive: timeout=3, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: text/html; charset=iso-8859-1

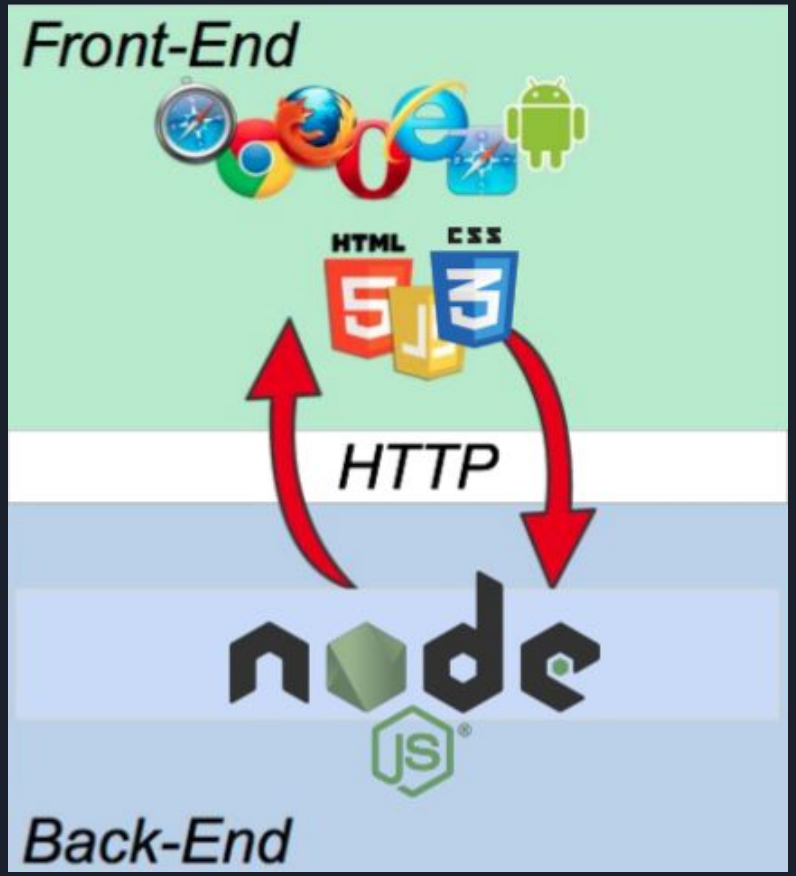
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>200 OK</title>
                ...
```

Google Data Center - Carolina do Sul



O Data Center é um ambiente projetado para abrigar servidores e outros componentes como sistemas de armazenamento de dados e ativos de rede (switches, roteadores, etc.)

Servidor usando o *Node.js*



Node.js

- É um interpretador de código JavaScript que funciona do lado do servidor
- Seu objetivo é ajudar programadores na criação de códigos capazes de manipular dezenas de milhares de conexões simultâneas, em uma única máquina física



- Foi criado por Ryan Dahl em 2009, e seu desenvolvimento é mantido pela empresa Joyent, onde Dahl trabalha

https://nodejs.org/en/

Pacotes



Node.js - Pacotes



- No Node.js podemos utilizar pacotes:
  - Pacotes são códigos em JavaScript prontos que lidam com certos tipos de domínios
- Existem diversos pacotes que já vêm com o Node.js
- Para utilizar os pacotes, utilizamos o comando require

```
require(“colors”);

console.log(“mensagem”.rainbow);
```

- O pacote colors fornece a função rainbow para imprimir colorido

Node.js - Pacotes



- Apesar do Node.js já vir com vários pacotes, existem muitos outros na Internet!

- Esses pacotes podem ser baixados por meio de um programa, chamado npm (*Node Package Manager*), que gerencia pacotes do Node.js

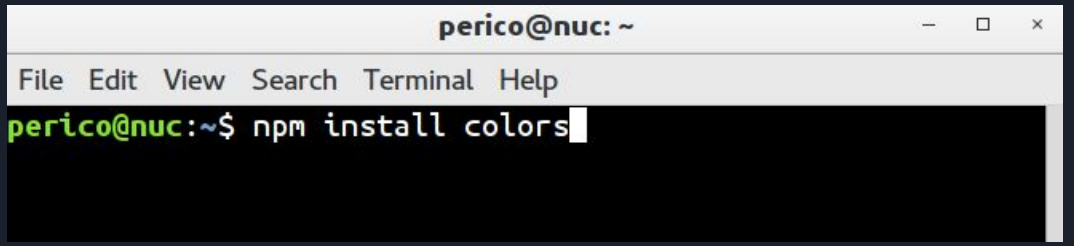


Node.js - Pacotes

- Para usar o npm, precisamos abrir o prompt do Windows, o terminal do Linux ou o terminal do WebStorm

- Para instalar um pacote utilizamos:





Node.js - Pacotes



- Alguns pacotes:
  - colors
  - express
  - url
  - http
  - fs



Node.js - Pacote Express

- Framework Express:
  - Express é aceita como a principal ferramenta para aplicações WEB pela maioria das comunidades Node.js
  - Contém um robusto conjunto de recursos para desenvolver aplicações WEB
    - Um exemplo, é o intuitivo sistema de *Views* (da arquitetura MVC)



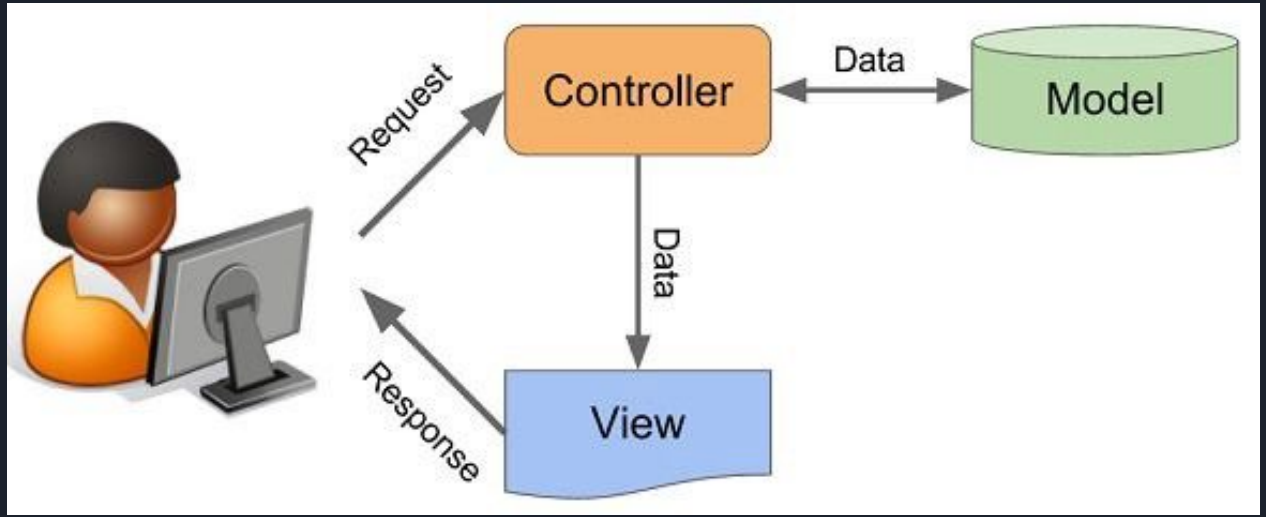
Parêntesis - Arquitetura MVC

Arquitetura MVC

- É um padrão de arquitetura de software
- MVC - Model-View-Controller
- Em português, Modelo-Visualização-Controlador
- A ideia do MVC é separar a lógica de negócio da interface com o usuário:
    - Assim, os programadores podem mudar facilmente cada parte, sem afetar as outras.

Arquitetura MVC

- É uma arquitetura das mais antigas e, também, uma das mais utilizadas na WEB

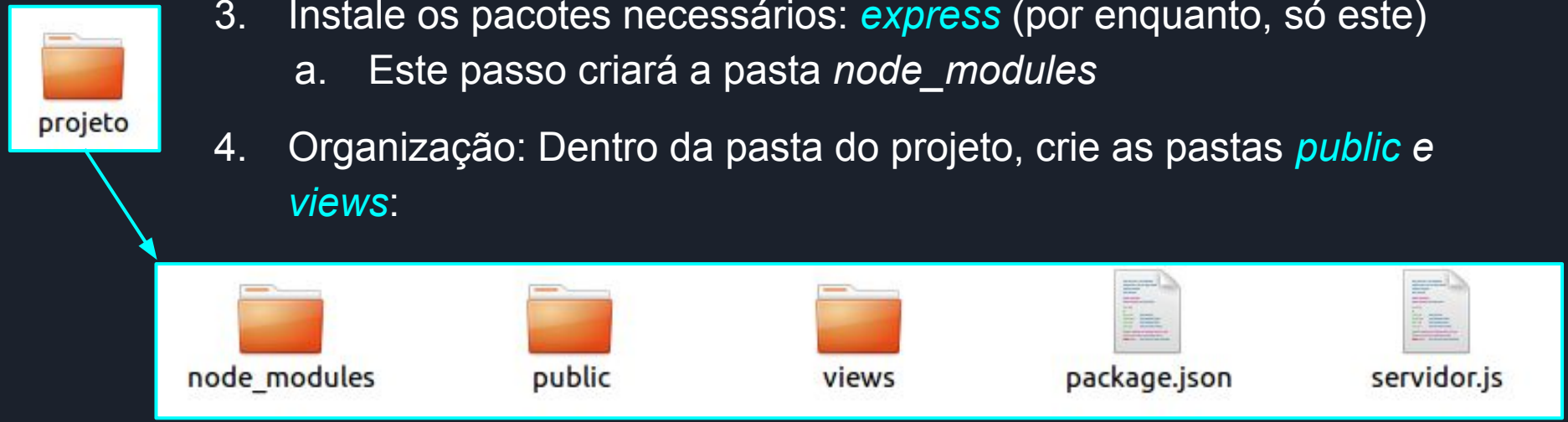


Fonte: https://medium.com/trainingcenter/mvc-framework-usando-a-arquitetura-sem-c%C3%B3digo-de-terceiros-bf95a744c66d

Fim dos Parêntesis

Servidor (Node.js + Express)
Como começar um projeto

1. Defina a pasta raiz do projeto: *servidor* (neste exemplo)
2. Utilize o comando *npm init* (no terminal, na pasta raiz) para criar um arquivo *package.json*
3. Instale os pacotes necessários: *express* (por enquanto, só este)
   a. Este passo criará a pasta *node_modules*
4. Organização: Dentro da pasta do projeto, crie as pastas *public* e *views*:



Node.js - Pacote Express



- O primeiro passo para utilizar o express é a instalação
- Digite no terminal:

```
npm install express --save
```

- Crie o arquivo *servidor.js*, que deve conter o código do servidor

Exemplo: Criando um servidor com *Node.js* e *Express*

```
1  // inclui o módulo http
2  var http = require('http');
3  // inclui o módulo express
4  var express = require('express');
5
6  // cria a variável app, pela qual acessaremos
7  // os métodos / funções existentes no framework
8  // express
9  var app = express();
10
11 // método use() utilizado para definir em qual
12 // pasta estará o conteúdo estático
13 app.use(express.static('./public'));
14
15 // cria o servidor
16 var server = http.createServer(app);
17
18 // define o número da porta que o servidor ouvirá
19 server.listen(3000);
20
21 // mensagem exibida no console para debug
22 console.log("servidor rodando...");
```

Conteúdo
Estático *vs.* Dinâmico

Estático *vs.* Dinâmico

Estático	Dinâmico
• Conteúdo inserido manualmente no código do site • Sem banco de dados • Sem Sistema Gerenciador de Conteúdo (*Content Management System* - CMS)	• Conteúdo inserido de maneira automática, por meio de scripts presentes no código do servidor (JavaScript, PHP, Python, Ruby, Java etc) • Com banco de dados • Normalmente, utiliza um Sistema Gerenciador de Conteúdo (*Content Management System* - CMS): WordPress, Joomla

Acessando as páginas pelo servidor

1. Abra o navegador
2. Para testar, você pode acessar o conteúdo disponibilizado pelo servidor (*com servidor e navegador no mesmo PC*) digitando o seguinte na barra de endereços:
   a. *localhost:porta/pagina.html*



3. Para acessar o conteúdo disponibilizado pelo servidor de outro computador, utilizamos o IP do PC que está rodando o servidor:
   a. *192.168.1.101:porta/pagina.html*

Git - .gitignore

Antes de dar o **commit** das modificações realizadas nessa aula. Vamos falar um pouco sobre o **.gitignore**.

O **.gitignore** é arquivo onde onde armazenamos uma lista de arquivos que dever ser ignorados no sistema de versionamento.

Como puderam perceber, alguns arquivos vistos na aula de hoje são gerados automaticamente. Nesse caso, não precisamos armazená-los no git. Portanto, iremos ignorar a pasta **node_modules** do nosso projeto.

Basta criar o arquivo **.gitignore** na pasta do projeto e salvar dentro dele o seguinte comando:

- node_modules/*

Vá além!

Leia mais sobre Servidor com NodeJS

https://www.w3schools.com/nodejs/default.asp

Não esqueçam de marcar presença!

Material desenvolvido por:

- Danilo Périco
- Isaac de Jesus
- Fagner Pimentel

Fontes:

- https://www.w3schools.com/